ILT Research System - Detailed Design Specification & Developer Guide (DDD v1.2 FINAL)

Engineering Blueprint for Coding: Analyst Replay, Dynamic Templates, Wire Protocol, DDL Schema & Developer Guide

Document Parameter	Specification Value	Document Parameter	Specification Value
Document ID:	ILT-DDD-V1.2-FINAL-EN	Publication Date:	October 2026 (Release 1.2 Final Edition)
Functional Baseline:	FRS v3.3 & Architecture Spec v5.3	Target Audience:	Software Engineers, Lead Architects, Full-Stack Developers
Analyst Capabilities:	Video-Synced Replay + Open-Source BI	Target Hardware:	Wacom MovinkPad 11 (Certified Continuous Charging)
Readiness Status:	Coding-Ready (Includes Docker Compose & Mock Stylus)	Timer Governance:	20s System Default (Director Auth Required)

1. Document Purpose, System Overview & Technology Stack

Complete engineering specification for coding implementation including the Research Analyst Portal and Kinematic Replay Engine.

System Layer	Technology	Version	Engineering Rationale
Analyst Portal (Replay & BI)	React, Canvas 2D Replayer, Apache Superset, DuckDB-Wasm	React 18+, DuckDB 1.0	133Hz stroke replay, video sync from MinIO, multi-dimensional cohort slicing, Parquet/Jupyter export.
Student Tablet (Kiosk App)	React, TypeScript, Native Canvas 2D, Wacom Web SDK, Dexie.js	React 18+, TS 5+	60 FPS rendering, 133Hz EMR sampling, IndexedDB resilience, continuous charging.
Proctor Cockpit Console	React, Canvas 80% Mirror, WebRTC DataChannel (Host), TailwindCSS	React 18+, TS 5+	Live stroke mirror (80%), telemetry (20%), 3 distinct metrics dashboard, classroom oversight.
Local Edge Gateway	Node.js / Fastify, WebSocket Signaling, go2rtc Proxy	Node 20 LTS	P2P direct channels, WebRTC signaling, agnostic video proxy (RTSP/ONVIF/USB).
Analytics & DTW Engine	Python, FastAPI, SciPy, FastDTW, NumPy, DuckDB	Python 3.11+	Kinematic signal processing, cDTW for Row 3, timer factoring queries.
Core Database & Storage	PostgreSQL 16 + TimescaleDB + MinIO S3	Postgres 16	Relational data, compressed TimescaleDB Hypertables, and MinIO S3 video storage.

2. Monorepo Project Structure & Module Organization

```
ilt-research-platform/
├── apps/
│   ├── student-tablet/          # אפליקציית הטאבלט לתלמיד (React + Canvas 2D + EMR Driver)
│   ├── proctor-cockpit/         # עמדת המאבחן (Control Page, Live Cockpit, Comparative
Dashboard)
│   └── analyst-portal/          # פורטל אנליסט (Kinematic Replayer, Desk Video Sync, Superset
Dashboards)
├── services/
│   ├── gateway/                 # שרת שער מקומי (Signaling, DB Cache, go2rtc Camera Proxy)
│   └── analytics/               # שירות ניתוח קינמטי, cDTW וייצוא לשורה 3, DuckDB/Parquet
(FastAPI)
├── packages/
│   ├── shared-types/            # מודלי נתונים, ממשקים ו-Enums משותפים (TypeScript)
│   ├── emr-protocol/            # מודול סריאליזציה בינארית (24-Byte Struct) ומילון OpCodes
│   └── template-schemas/        # סכמות JSON של תבניות כתיבה (3 גריד x5, מחברת 10 שורות,
אותיות ט')
└── docker/                      # תצורות Docker לפיתוח מקומי (Postgres+Timescale, MinIO,
go2rtc)
```

3. Binary Wire Protocol & Serialization Specification

```
// C / C++ & TypedArray 24-Byte Binary EMR Sample Struct (Little-Endian IEEE 754)
#pragma pack(push, 1)
struct EMRSampleRecord {
    uint32_t timestamp_offset_ms; // 4 Bytes: Milliseconds elapsed since block launch
    float    x_coord;             // 4 Bytes: Normalized X sub-pixel coordinate (0.0000 - 1.0000)
    float    y_coord;             // 4 Bytes: Normalized Y sub-pixel coordinate (0.0000 - 1.0000)
    float    pressure;            // 4 Bytes: Normalized pen tip pressure (0.0000 - 1.0000)
    int16_t  tilt_altitude;       // 2 Bytes: Pen vertical angle relative to tablet plane (0 - 90
deg)
    int16_t  tilt_azimuth;        // 2 Bytes: Pen rotational angle around surface normal (0 - 360
deg)
    uint8_t  stroke_flags;        // 1 Byte: Bit 0: PenDown, Bit 1: InAirHover, Bit 2: Eraser, Bit
3: TargetHit
    uint8_t  active_house_id;     // 1 Byte: Identifier of active house/cell (1 to 15)
    uint16_t reserved_padding;    // 2 Bytes: Alignment padding (0x0000)
};
#pragma pack(pop)
```

4. Dynamic Canvas Engine & Templates Specification

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "ILTDynamicTemplateDefinition",
  "type": "object",
  "properties": {
    "template_id": { "type": "string" },
    "template_name": { "type": "string" },
    "ruling_type": { "type": "string", "enum": ["NOTEBOOK_10_ROWS", "NOTEBOOK_14_ROWS",
"NOTEBOOK_1_ROW", "GRID_3X5", "FREE_CANVAS"] },
    "target_grade_level": { "type": "string", "default": "ALL_GRADES" },
```

```
    "canvas_schema": {
      "type": "object",
      "properties": {
        "num_lines": { "type": "integer" },
        "line_spacing_px": { "type": "number" },
        "margin_side": { "type": "string", "enum": ["RIGHT", "LEFT", "NONE"] },
        "margin_offset_px": { "type": "number" },
        "active_row_indices": { "type": "array", "items": { "type": "integer" } }
      },
      "required": ["num_lines", "line_spacing_px", "margin_side", "active_row_indices"]
    },
    "rules_schema": {
      "type": "object",
      "properties": {
        "max_internal_pen_lifts": { "type": "integer", "default": 0 },
        "width_nominal_mm": { "type": "number", "default": 15.0 },
        "width_tolerance_mm": { "type": "number", "default": 2.0 },
        "forward_only_locking": { "type": "boolean", "default": true }
      }
    }
  },
  "required": ["template_id", "template_name", "ruling_type", "canvas_schema", "rules_schema"]
}
```

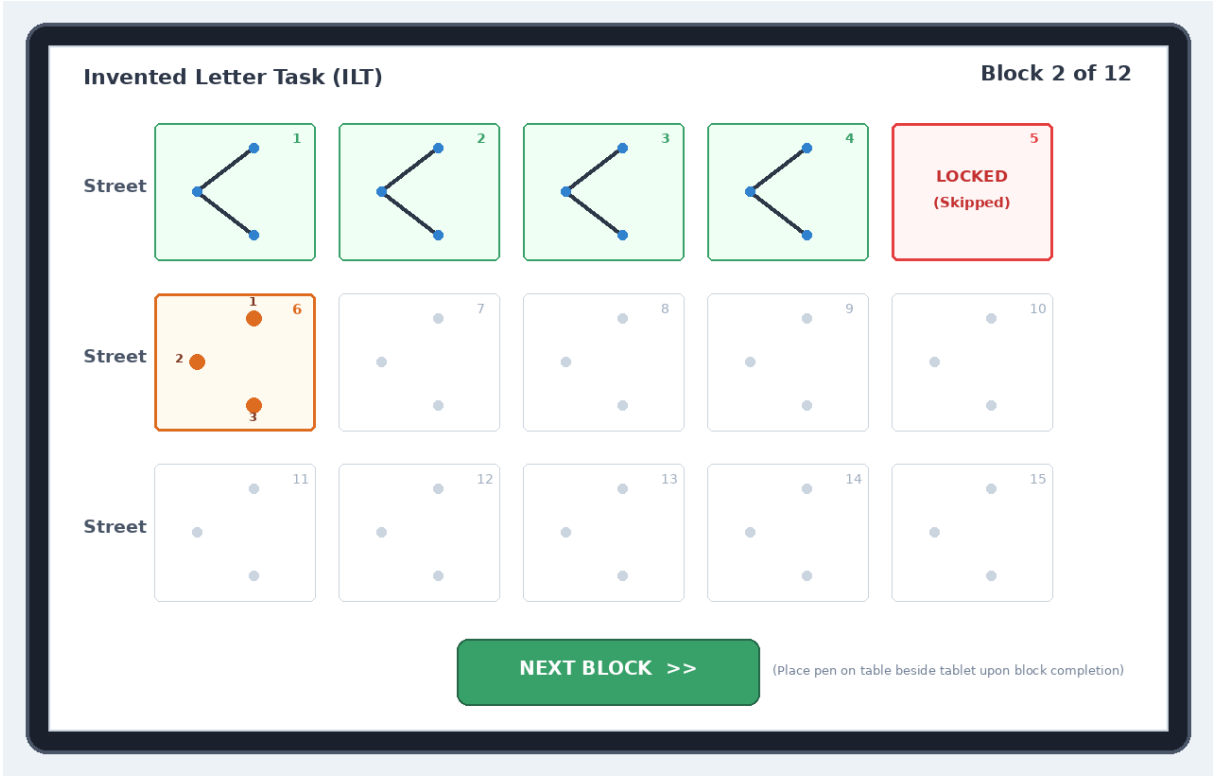


Figure 1: Student Tablet Canvas with Forward-Only Dynamic Lock

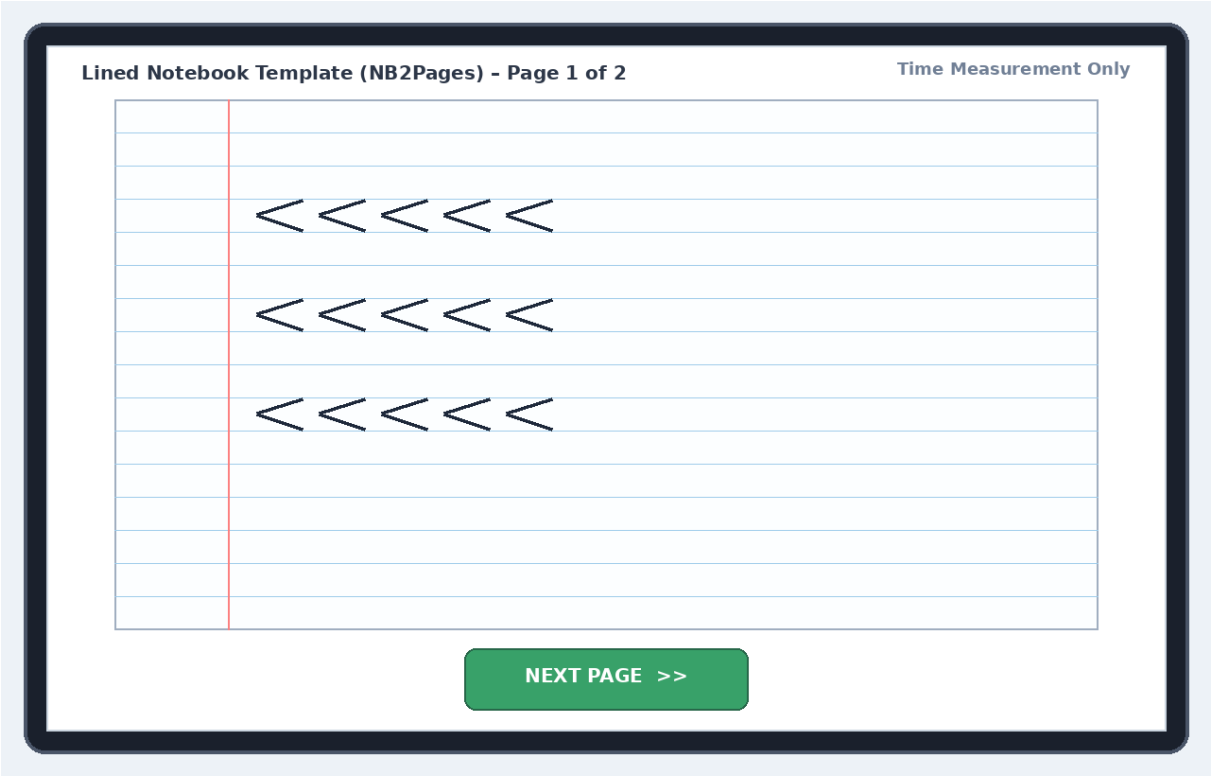


Figure 2: Lined Notebook Canvas Aligned to Original Study Ruling

5. State Machines, Proctor Cockpit & Comparative Dashboard

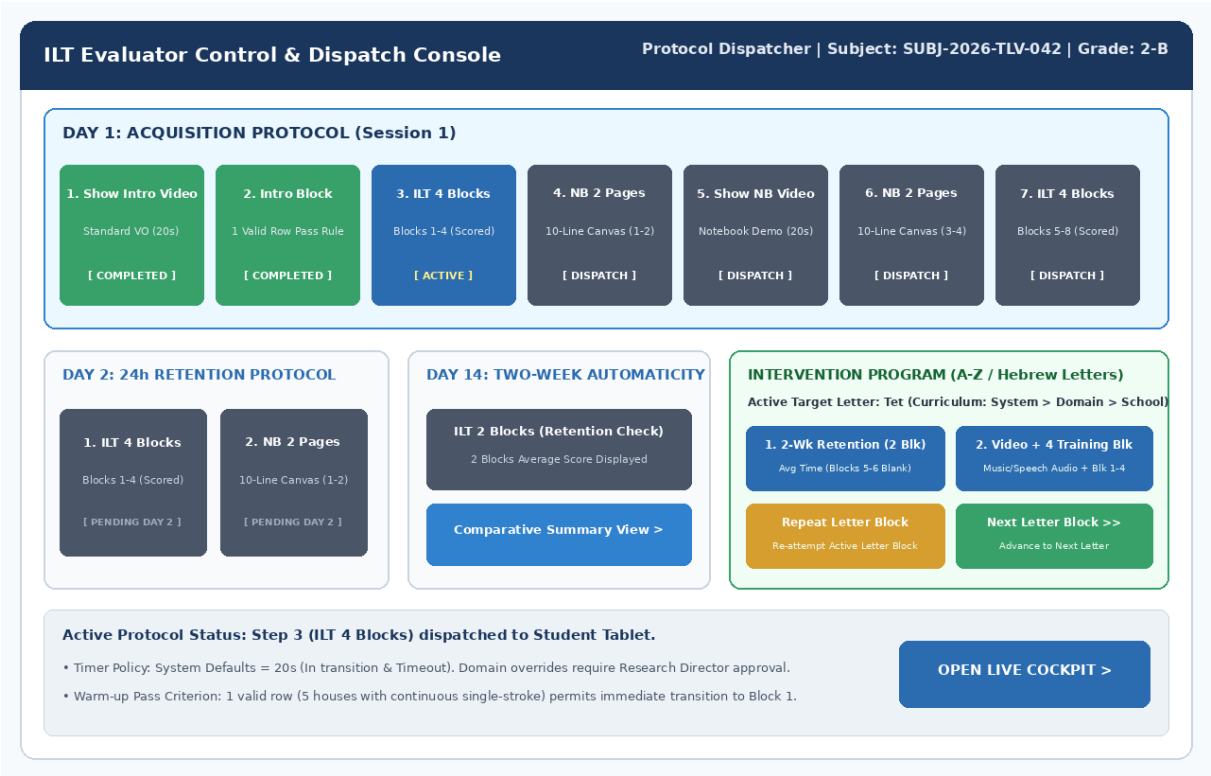


Figure 3: Evaluator Control & Protocol Dispatcher Console



Figure 4: Proctor Cockpit (80% Live Mirror / 20% Telemetry)

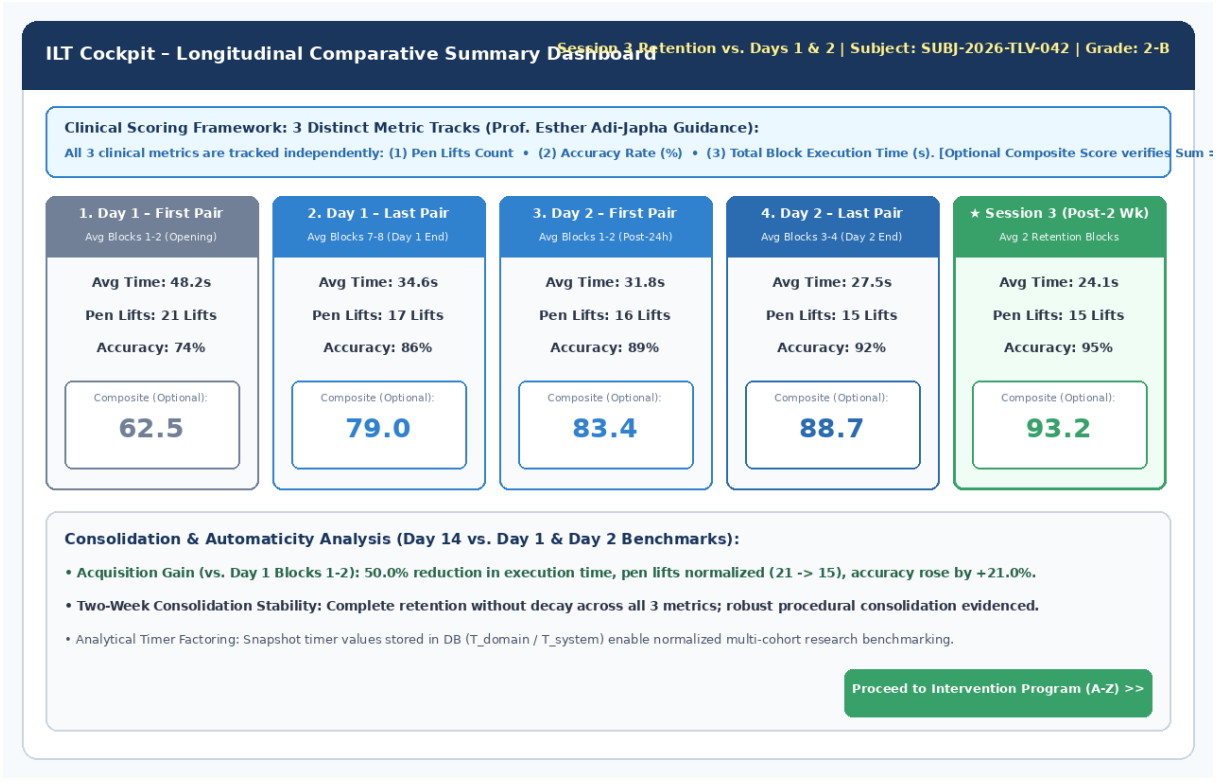


Figure 5: Longitudinal Comparative Summary Dashboard (3 Distinct Metrics)

6. Relational Database Schema & Analytical Views (DDL v5.3)

```
-- ================================================================================
-- ILT PRODUCTION DDL (PostgreSQL 16 + TimescaleDB) - Release 5.3 / DDD v1.2
```

```
-- ============================================================================
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";
CREATE EXTENSION IF NOT EXISTS "timescaledb";

-- 1. System Configuration & Global Defaults
CREATE TABLE system_config (
    config_id INT PRIMARY KEY DEFAULT 1,
    in_transition_delay_sec INT NOT NULL DEFAULT 20, -- Default In-Transition: 20s
    inactivity_timeout_sec INT NOT NULL DEFAULT 20,  -- Default Inactivity Timeout: 20s
    post_block_timeout_sec INT NOT NULL DEFAULT 30,  -- Timeout post-15th letter: 30s
    notebook_page_timeout_sec INT NOT NULL DEFAULT 60,
    video_recording_enabled BOOLEAN NOT NULL DEFAULT TRUE,
    require_research_director_approval BOOLEAN NOT NULL DEFAULT TRUE,
    score_weight_pen_lifts_pct NUMERIC(5,2) NOT NULL DEFAULT 35.00,
    score_weight_accuracy_pct NUMERIC(5,2) NOT NULL DEFAULT 40.00,
    score_weight_time_pct NUMERIC(5,2) NOT NULL DEFAULT 25.00,
    CONSTRAINT chk_weights_sum_100 CHECK (
        (score_weight_pen_lifts_pct + score_weight_accuracy_pct + score_weight_time_pct) = 100.00
    ),
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 2. Domains Table (with Research Director Authorization for Overrides)
CREATE TABLE domains (
    domain_id SERIAL PRIMARY KEY,
    domain_name VARCHAR(100) NOT NULL UNIQUE,
    override_in_transition_sec INT NULL,
    override_inactivity_timeout_sec INT NULL,
    override_video_enabled BOOLEAN NULL,
    timer_approved_by_research_director VARCHAR(100) NULL,
    timer_approval_timestamp TIMESTAMP WITH TIME ZONE NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 3. Dynamic Templates Engine
CREATE TABLE dynamic_templates (
    template_id VARCHAR(50) PRIMARY KEY,
    template_name VARCHAR(100) NOT NULL,
    template_category VARCHAR(30) NOT NULL,
    ruling_type VARCHAR(30) NOT NULL,
    target_grade_level VARCHAR(30) DEFAULT 'ALL_GRADES',
    canvas_schema JSONB NOT NULL,
    rules_schema JSONB NOT NULL,
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 4. Canonical Models for Freehand Row 3 Evaluation
CREATE TABLE letter_canonical_models (
    letter_id VARCHAR(10) PRIMARY KEY,
    language_code VARCHAR(5) NOT NULL,
    letter_char VARCHAR(5) NOT NULL,
    expected_strokes_count INT NOT NULL DEFAULT 1,
    canonical_aspect_ratio NUMERIC(5,3) NOT NULL,
```

```
    canonical_points JSONB NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 5. Sessions Table (3 Metrics Preserved Independently + Timer Snapshots)
CREATE TABLE sessions (
    session_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    session_number BIGSERIAL UNIQUE,
    domain_id INT NOT NULL REFERENCES domains(domain_id),
    school_name VARCHAR(150) NOT NULL,
    grade_class VARCHAR(50) NOT NULL,
    evaluator_name VARCHAR(120) NOT NULL,
    assessment_date DATE NOT NULL DEFAULT CURRENT_DATE,
    examinee_alias VARCHAR(50) NOT NULL,
    examinee_initials VARCHAR(5) NOT NULL,
    student_language VARCHAR(5) DEFAULT 'he',
    session_stage VARCHAR(30) NOT NULL,
    is_charging_during_test BOOLEAN DEFAULT FALSE,
    sys_in_transition_sec INT NOT NULL DEFAULT 20,
    dom_in_transition_sec INT NOT NULL DEFAULT 20,
    sys_inactivity_sec INT NOT NULL DEFAULT 20,
    dom_inactivity_sec INT NOT NULL DEFAULT 20,
    video_recording_enabled BOOLEAN NOT NULL DEFAULT TRUE,
    warmup_row_passed BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 6. Assessment Blocks Table (Preserving 3 Distinct Metrics Independently)
CREATE TABLE session_blocks (
    block_id BIGSERIAL PRIMARY KEY,
    session_id UUID NOT NULL REFERENCES sessions(session_id) ON DELETE CASCADE,
    block_type VARCHAR(30) NOT NULL,
    block_index INT NOT NULL,
    target_letter VARCHAR(5) NULL,
    pen_lifts_count INT NOT NULL,
    accuracy_pct NUMERIC(5,2) NOT NULL,
    total_duration_ms INT NOT NULL,
    composite_score NUMERIC(6,2) NULL,
    is_repeated_disrupted BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 7. High-Frequency EMR Telemetry (TimescaleDB Hypertable with 1-Day Chunks)
CREATE TABLE pen_telemetry_chunks (
    recorded_at TIMESTAMP WITH TIME ZONE NOT NULL,
    session_id UUID NOT NULL,
    block_index INT NOT NULL,
    house_id INT NOT NULL,
    time_offset_ms INT NOT NULL,
    x NUMERIC(8,4) NOT NULL,
    y NUMERIC(8,4) NOT NULL,
    pressure NUMERIC(6,5) NOT NULL,
    tilt_altitude SMALLINT NOT NULL,
    tilt_azimuth SMALLINT NOT NULL,
```

```
    is_hover BOOLEAN NOT NULL,
    is_pen_down BOOLEAN NOT NULL
);
SELECT create_hypertable('pen_telemetry_chunks', 'recorded_at', chunk_time_interval => INTERVAL '1
day');
ALTER TABLE pen_telemetry_chunks SET (timescaledb.compress, timescaledb.compress_segmentby =
'session_id, block_index');
SELECT add_compression_policy('pen_telemetry_chunks', INTERVAL '7 days');

-- 8. Video Replay Records (Desk & Hands RTSP / MP4 Ingestion)
CREATE TABLE session_video_records (
    video_id BIGSERIAL PRIMARY KEY,
    session_id UUID NOT NULL REFERENCES sessions(session_id) ON DELETE CASCADE,
    camera_model VARCHAR(100) DEFAULT 'Wireless IP Camera (RTSP)',
    storage_bucket VARCHAR(50) DEFAULT 'ilt-session-videos',
    video_object_key TEXT NOT NULL,
    duration_sec NUMERIC(8,2) NOT NULL,
    start_time_utc TIMESTAMP WITH TIME ZONE NOT NULL,
    fps NUMERIC(4,2) DEFAULT 25.00,
    resolution VARCHAR(20) DEFAULT '1280x720',
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 9. Analytical Cohort Slices View (for Apache Superset, Metabase, DuckDB, Jupyter)
CREATE OR REPLACE VIEW v_analyst_cohort_slices AS
SELECT
    s.session_id,
    s.session_number,
    s.examinee_alias,
    s.examinee_initials,
    d.domain_name,
    s.school_name,
    s.grade_class,
    s.dominant_hand,
    s.student_language,
    s.session_stage,
    s.assessment_date,
    s.sys_in_transition_sec,
    s.dom_in_transition_sec,
    (s.dom_in_transition_sec::numeric / s.sys_in_transition_sec::numeric) AS timer_factoring_ratio,
    sb.block_id,
    sb.block_type,
    sb.block_index,
    sb.target_letter,
    sb.pen_lifts_count,
    sb.accuracy_pct,
    sb.total_duration_ms,
    (sb.total_duration_ms::numeric / 1000.0) AS total_duration_sec,
    ((sb.total_duration_ms::numeric / 1000.0) * (s.sys_in_transition_sec::numeric /
s.dom_in_transition_sec::numeric)) AS factored_duration_sec,
    sb.composite_score,
    sb.is_repeated_disrupted
FROM sessions s
```

```
JOIN domains d ON s.domain_id = d.domain_id
JOIN session_blocks sb ON s.session_id = sb.session_id;
```

7. Kinematic Session Replay Engine & Cohort Analytics Implementation

- **TypeScript Kinematic Replay Engine Implementation:** Canvas 2D stroke replayer, variable speed controls (0.25x-4x), and synchronized desk video:

```
// Kinematic Session Replay Engine (TypeScript / React Canvas 2D)
export class KinematicSessionReplayer {
    private currentFrameIndex = 0;
    private playbackSpeed = 1.0;
    private isPlaying = false;
    private animFrameId: number | null = null;

    constructor(
        private canvas: HTMLCanvasElement,
        private samples: any[],
        private videoElement?: HTMLVideoElement,
        private onProgress?: (timeMs: number, sample: any) => void
    ) {}

    public play(): void {
        this.isPlaying = true;
        if (this.videoElement) this.videoElement.play();
        this.renderLoop();
    }

    public pause(): void {
        this.isPlaying = false;
        if (this.animFrameId) cancelAnimationFrame(this.animFrameId);
        if (this.videoElement) this.videoElement.pause();
    }

    public setSpeed(speed: number): void {
        this.playbackSpeed = speed;
        if (this.videoElement) this.videoElement.playbackRate = speed;
    }

    public seek(targetTimeMs: number): void {
        this.currentFrameIndex = this.samples.findIndex(s => s.timestamp_offset_ms >=
targetTimeMs);
        if (this.currentFrameIndex === -1) this.currentFrameIndex = this.samples.length - 1;
        if (this.videoElement) this.videoElement.currentTime = targetTimeMs / 1000.0;
        this.renderUpToCurrentFrame();
    }

    private renderLoop = (): void => {
        if (!this.isPlaying) return;
        const currentSample = this.samples[this.currentFrameIndex];
        this.drawStrokeSegment(currentSample);
        if (this.onProgress) this.onProgress(currentSample.timestamp_offset_ms, currentSample);

        this.currentFrameIndex += Math.max(1, Math.round(this.playbackSpeed));
        if (this.currentFrameIndex < this.samples.length) {
```

```
            this.animFrameId = requestAnimationFrame(this.renderLoop);
        } else {
            this.pause();
        }
    };

    private drawStrokeSegment(s: any): void {
        const ctx = this.canvas.getContext('2d')!;
        const x = s.x_coord * this.canvas.width;
        const y = s.y_coord * this.canvas.height;
        if (s.stroke_flags & 0x01) { // Pen Down
            ctx.fillStyle = this.getPressureColor(s.pressure);
            ctx.beginPath();
            ctx.arc(x, y, 2 + s.pressure * 4, 0, Math.PI * 2);
            ctx.fill();
        } else if (s.stroke_flags & 0x02) { // In-Air Hover
            ctx.fillStyle = '#ECC94B'; // Yellow hover line
            ctx.fillRect(x - 1, y - 1, 2, 2);
        }
    }

    private getPressureColor(p: number): string {
        if (p < 0.20) return '#38A169'; // Light Green
        if (p < 0.40) return '#3182CE'; // Medium Blue
        return '#E53E3E';               // Firm Red
    }
}
```

- **Python DuckDB / Parquet Cohort Export Endpoint:** High-speed cohort slicing and direct Parquet export for researchers:

```
# Python Cohort Analytics & Parquet / DuckDB Export Engine (FastAPI)
import duckdb
import pandas as pd
from fastapi import APIRouter, Response

router = APIRouter(prefix="/api/v1/analyst")

@router.get("/export/parquet")
def export_cohort_parquet(domain: str = None, grade: str = None, ses_low: bool = None):
    con = duckdb.connect()
    con.execute("INSTALL postgres; LOAD postgres;")
    con.execute("ATTACH 'dbname=ilt_research user=ilt_admin host=db' AS pg (TYPE POSTGRES);")

    query = "SELECT * FROM pg.v_analyst_cohort_slices WHERE 1=1"
    if domain: query += f" AND domain_name = '{domain}'"
    if grade: query += f" AND grade_class LIKE '%{grade}%'"

    df = con.execute(query).df()
    parquet_bytes = df.to_parquet(index=False)

    return Response(
        content=parquet_bytes,
        media_type="application/octet-stream",
```

```
    headers={"Content-Disposition": "attachment; filename=ilt_cohort_data.parquet"}
)
```

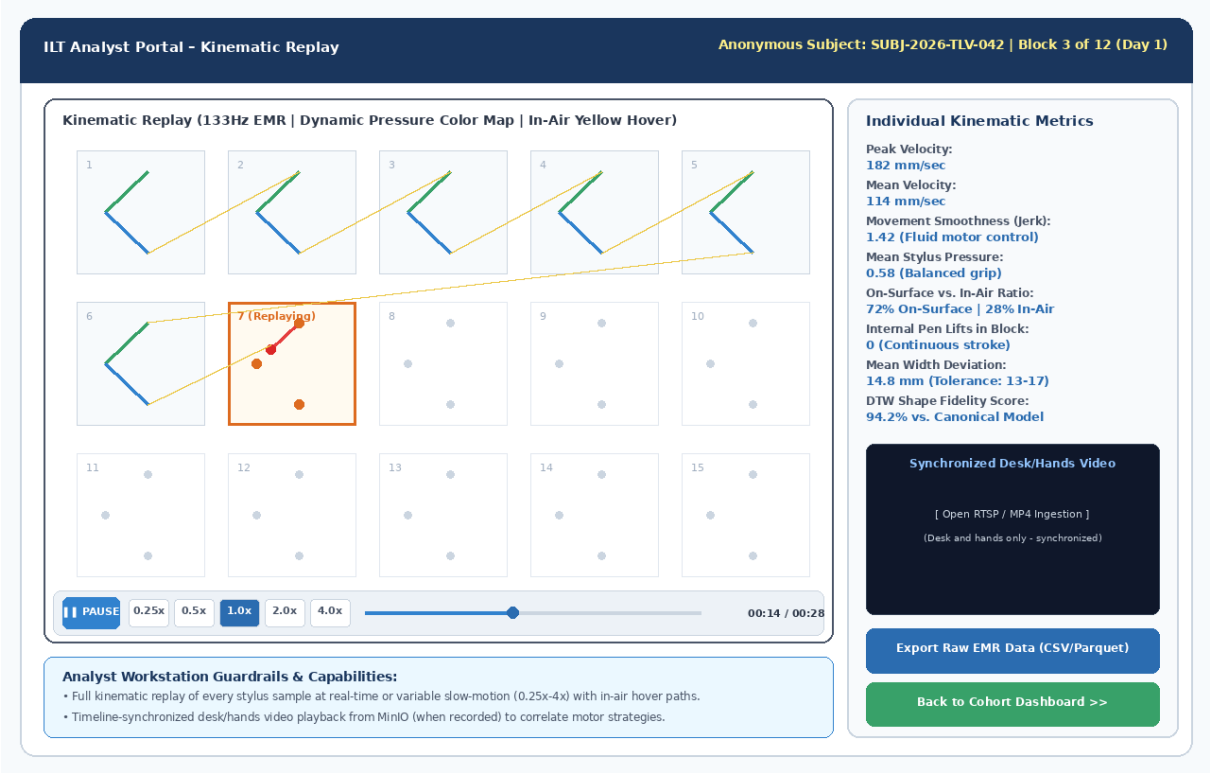


Figure 6: Analyst Portal – Kinematic Session Replay Engine Synced with Desk Video

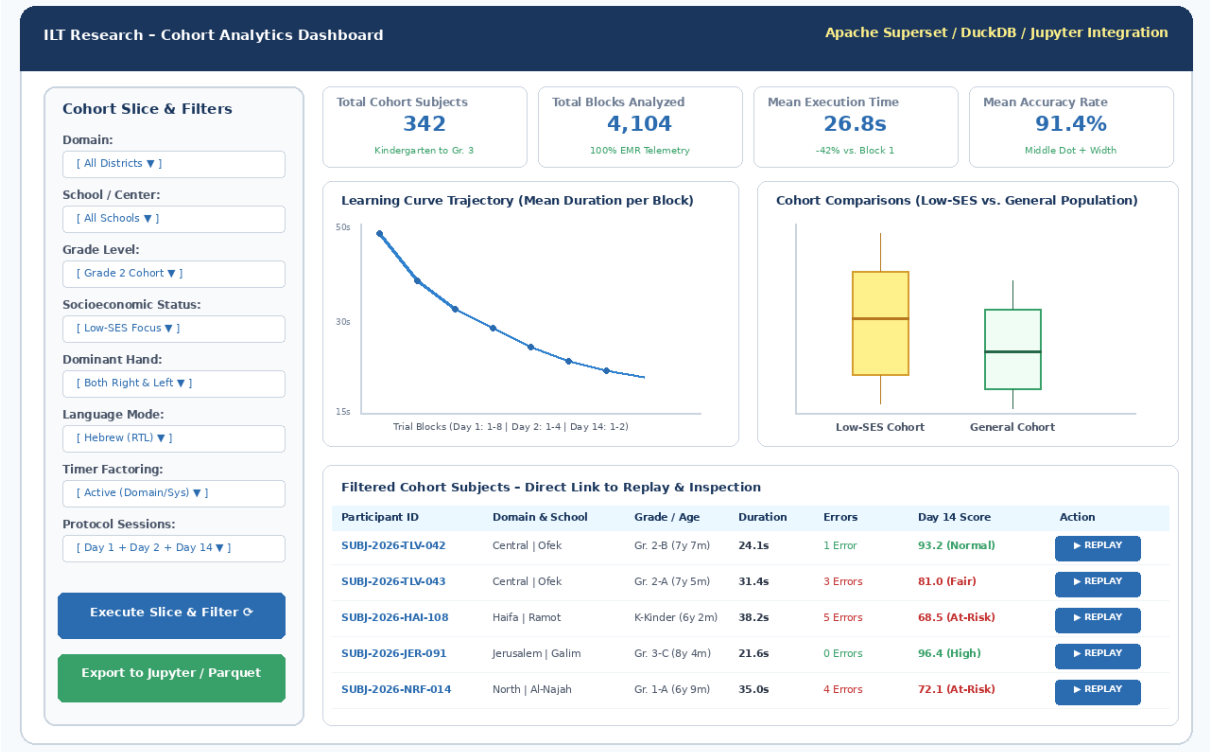


Figure 7: Open-Source Cohort Slice-and-Dice Analytics Dashboard

8. Kinematic Algorithms & Intervention Program

```
import numpy as np
from scipy.spatial import procrustes
from fastdtw import fastdtw

def evaluate_freehand_row3(stroke_points: np.ndarray, canonical_points: np.ndarray,
canonical_aspect_ratio: float) -> dict:
    if len(stroke_points) < 10:
        return {"composite_accuracy": 0.0, "diagnostic_tag": "EMPTY_OR_NOISE", "valid": False}

    diffs = np.diff(stroke_points, axis=0)
    dists = np.hypot(diffs[:, 0], diffs[:, 1])
    cum_dist = np.insert(np.cumsum(dists), 0, 0)
    total_len = cum_dist[-1]
    even_spaces = np.linspace(0, total_len, 100)
    resampled_stroke = np.column_stack((
        np.interp(even_spaces, cum_dist, stroke_points[:, 0]),
        np.interp(even_spaces, cum_dist, stroke_points[:, 1])
    ))

    norm_stroke = (resampled_stroke - np.mean(resampled_stroke, axis=0)) /
(np.std(resampled_stroke) + 1e-6)
    norm_canonical = (canonical_points - np.mean(canonical_points, axis=0)) /
(np.std(canonical_points) + 1e-6)

    _, _, disparity = procrustes(norm_canonical, norm_stroke)
    procrustes_score = max(0.0, min(100.0, 100.0 * (1.0 - disparity * 2.2)))

    dtw_dist, _ = fastdtw(norm_stroke, norm_canonical, dist=2)
    dtw_score = max(0.0, min(100.0, 100.0 * (1.0 - (dtw_dist / 100.0))))

    w, h = np.ptp(stroke_points[:, 0]), np.ptp(stroke_points[:, 1])
    actual_ratio = w / (h + 1e-6)
    ratio_dev_pct = abs(actual_ratio - canonical_aspect_ratio) / canonical_aspect_ratio * 100.0

    composite_acc = 0.55 * procrustes_score + 0.30 * dtw_score + 0.15 * max(0.0, 100.0 -
ratio_dev_pct)
    tag = "ACCURATE" if composite_acc >= 80.0 else ("PROPORTION_DISTORTION" if ratio_dev_pct > 25.0
else "SHAPE_DEFORMATION")

    return {
        "composite_accuracy": round(composite_acc, 2),
        "procrustes_score": round(procrustes_score, 2),
        "dtw_distance": round(dtw_dist, 2),
        "aspect_ratio_deviation_pct": round(ratio_dev_pct, 2),
        "diagnostic_tag": tag,
        "valid": True
    }
```

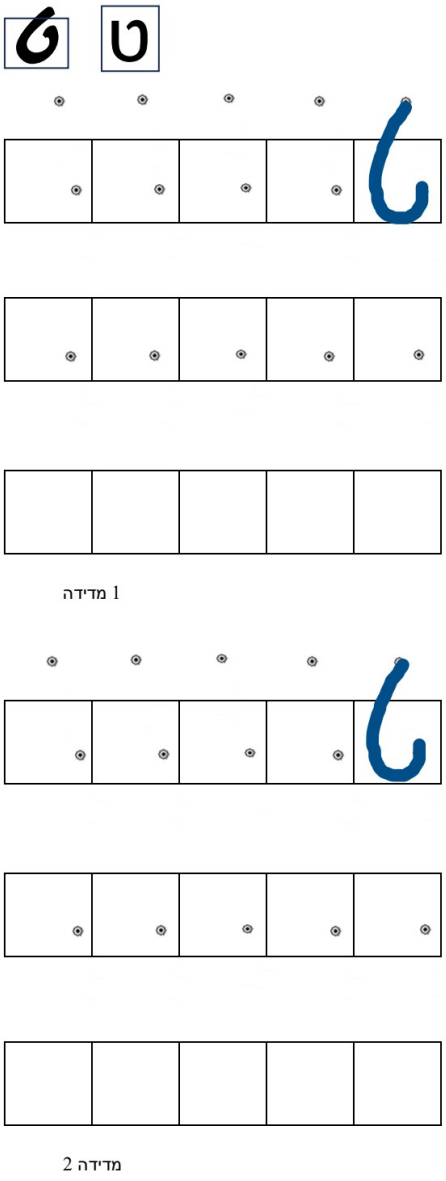


Figure 8: Sample Graded Intervention Training Sheet (Hebrew Letter Tet)

9. Appendix: Developer Implementation & Onboarding Guide

- **Docker Compose Local Stack:** Launch TimescaleDB, MinIO, and go2rtc locally with one command:

```
version: '3.8'
services:
  db:
    image: timescale/timescaledb:latest-pg16
    container_name: ilt_db
    environment:
      POSTGRES_DB: ilt_research
      POSTGRES_USER: ilt_admin
      POSTGRES_PASSWORD: ilt_secret_password
    ports:
      - "5432:5432"
    volumes:
```

```
      - pgdata:/var/lib/postgresql/data
    restart: always

  minio:
    image: minio/minio:latest
    container_name: ilt_minio
    command: server /data --console-address ":9001"
    environment:
      MINIO_ROOT_USER: minio_admin
      MINIO_ROOT_PASSWORD: minio_secret_password
    ports:
      - "9000:9000"
      - "9001:9001"
    volumes:
      - miniodata:/data
    restart: always

  go2rtc:
    image: alexxit/go2rtc:latest
    container_name: ilt_cms_gateway
    ports:
      - "1984:1984" # Management & WebRTC API
      - "8554:8554" # RTSP Relay Port
      - "8555:8555" # WebRTC Live Port
    volumes:
      - ./go2rtc.yaml:/config/go2rtc.yaml
    restart: always

volumes:
  pgdata:
  miniodata:
```

- **Mock Stylus Simulator for Desktop Development:** Maps mouse events to simulated EMR samples with pressure and hover:

```
// Mock Wacom EMR Stylus Simulator for Desktop Browser Development
export class MockEMRStylusSimulator {
    private isMouseDown = false;
    private startTime = 0;

    constructor(private canvas: HTMLCanvasElement, private onSample: (s: any) => void) {
        this.canvas.addEventListener('mousedown', (e) => {
            this.isMouseDown = true;
            this.startTime = performance.now();
            this.emit(e, 0.45, true, false);
        });
        window.addEventListener('mouseup', (e) => {
            if (this.isMouseDown) {
                this.isMouseDown = false;
                this.emit(e, 0.0, false, false);
            }
        });
        this.canvas.addEventListener('mousemove', (e) => {
            const pressure = this.isMouseDown ? 0.55 + 0.15 * Math.sin(performance.now() / 80) :
0.0;
```

```
            const isHover = !this.isMouseDown;
            this.emit(e, pressure, this.isMouseDown, isHover);
        });
    }

    private emit(e: MouseEvent, pressure: number, isPenDown: boolean, isHover: boolean) {
        const rect = this.canvas.getBoundingClientRect();
        const x = (e.clientX - rect.left) / rect.width;
        const y = (e.clientY - rect.top) / rect.height;
        this.onSample({
            time_offset_ms: Math.round(performance.now() - this.startTime),
            x: Number(x.toFixed(5)),
            y: Number(y.toFixed(5)),
            pressure: Number(pressure.toFixed(4)),
            tilt_altitude: 60,
            tilt_azimuth: 180,
            is_pen_down: isPenDown,
            is_hover: isHover
        });
    }
}
```

- **Mandatory Developer Guardrails:** (1) Strictly 24-byte binary protocol for live EMR telemetry; (2) Enforce Sum(Weights) == 100% in UI and API; (3) Research Director authorization code required for timer overrides; (4) Preserve 3 distinct metric columns across all APIs and exports.